



Final Project

Mushroom Classification: Predicting Edibility

By:

Hiba Slaymen: 202203516

Fatima Srour: 202202944

(hiba.slaymen@lau.edu)

(fatima.srour01@lau.edu)

Submitted to Dr. Abbas Rammal as Part of the Course

Introduction to Machine Learning

On

December 8th, 2024

Table of Contents:

I. Collecting Data

II. Preparing the Data

- **Data Cleaning**
- **Data Splitting**

III. Data Visualization

IV. Choosing a Model

- **Random Forest Tree**
- **Linear Regression**
- **Naïve Bayes**

V. Training and Testing

VI. Evaluation

VII. Cross Validation

VIII. Hyperparameter Tuning

IX. Validation Testing

I. Collecting Data

Link of dataset: <https://www.kaggle.com/prishasawhney/mushroom-dataset>

The mushroom dataset has 54035 entries from Kaggle used in this analysis provides detailed information about various physical attributes of mushrooms, such as cap shape, gill attachment, and stem characteristics, with the goal of classifying them as either edible or poisonous.

The dataset contains the following columns:

1. **cap-diameter**: The diameter of the mushroom cap.
2. **cap-shape**: Feature indicating the shape of the mushroom cap.
3. **gill-attachment**: Feature indicating how the gills are attached to the mushroom.
4. **gill-color**: Feature indicating the color of the gills.
5. **stem-height**: Feature indicating the height of the mushroom stem.
6. **stem-width**: Feature indicating the width of the mushroom stem.
7. **stem-color**: Feature indicating the color of the stem.
8. **season**: Feature representing the season
9. **class**: The target variable, indicating whether the mushroom is edible (1) or poisonous (0).

II. Preparing the Data

➤ Data Cleaning:

1. Remove duplicate rows: There are 303 duplicate rows in the dataset, which were removed.
2. Check for missing value:
 - a. For numerical attributes with a small number of missing values, replace the missing values with the mean.
 - b. For categorical attributes, or numerical attributes with a high standard deviation, replace the missing values with the mode.
 - c. Rows with missing values for critical quality-related attributes should be dropped, as predicting quality based on incomplete quality classes is inefficient; quality prediction should rely on combinations of other attributes.
 - d. If a large portion of a column's data is missing, it may be best to drop the entire column, especially if it is not essential for the analysis.
3. Correct data format: This stage of data cleaning focuses on ensuring that all data is correctly formatted, whether as integers, floats, text, or other types.

```

df_info = df.info()
missing_values = df.isnull().sum()
duplicates = df.duplicated().sum()
df_types = df.dtypesdf_info, missing_values, duplicates, df_types

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 54035 entries, 0 to 54034
Data columns (total 9 columns):
#   Column              Non-Null Count  Dtype
---  -
0   cap-diameter         54035 non-null  int64
1   cap-shape            54035 non-null  int64
2   gill-attachment      54035 non-null  int64
3   gill-color           54035 non-null  int64
4   stem-height          54035 non-null  float64
5   stem-width           54035 non-null  int64
6   stem-color           54035 non-null  int64
7   season               54035 non-null  float64
8   class                54035 non-null  int64
dtypes: float64(2), int64(7)
memory usage: 3.7 MB
(None,
 cap-diameter      0
 cap-shape         0
 gill-attachment   0
 gill-color        0
 stem-height       0
 stem-width        0
 stem-color        0
 season           0
 class            0
 dtype: int64,
 303,
 cap-diameter      int64
 cap-shape         int64
 gill-attachment   int64
 gill-color        int64
 stem-height       float64
 stem-width        int64
 stem-color        int64
 season           float64
 class            int64
 dtype: object)

```

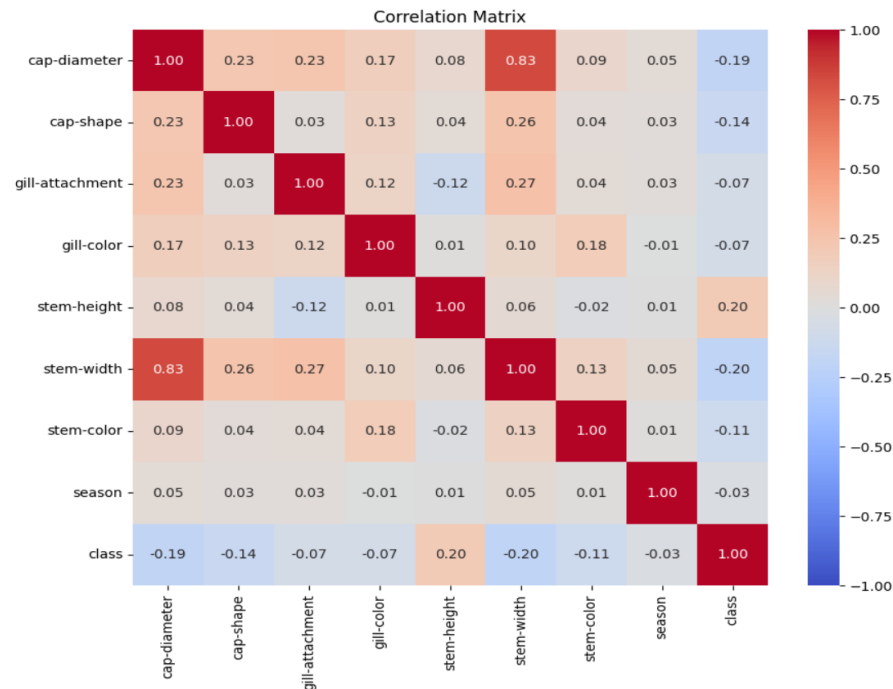
4. Finding and Removing Outliers:

Outliers need to be found and removed to reduce noise and improve the model's accuracy.



We used a boxplot to detect outliers (see the Collab notebook).

5. **Analyzing Correlations:** In this step I selected the numerical columns, calculated the correlation matrix and visualized it using a heatmap to identify relationships between the variables.
6. **Normalization:** Normalization is used to scale numerical data to a standard range, which is important for models sensitive to feature scaling. However, in our case, since we are using the Random Forest model, normalization is not required. Random Forest is an ensemble method based on decision trees, which are not sensitive to the scale of the data. Therefore, no normalization was applied.



➤ Data Splitting:

- ✓ Data splitting is a crucial step in machine learning, ensuring that a model is trained and evaluated properly. It involves dividing the dataset into different subsets: one for training and the other for testing.
- ✓ How to Split Data: You can split your data using `train_test_split` from `sklearn.model_selection`. It takes the features (X) and the target (y) and returns the training and testing sets.

```
from sklearn.model_selection import train_test_split

# Splitting the data into features (X) and target (y)
X = df.drop(columns=['class']) # Features
y = df['class'] # Target variable

# Splitting the data into a training set (80%) and a testing set (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Displaying the shapes of the resulting sets
print("Training set shape: ", X_train.shape, y_train.shape)
print("Testing set shape: ", X_test.shape, y_test.shape)
```

Training set shape: (35048, 8) (35048,)
 Testing set shape: (8763, 8) (8763,)

- X represents the features (independent variables).
- y represents the target variable (dependent variable).
- `train_test_split` splits the data into training (80%) and testing (20%) sets.
- `random_state=42` ensures reproducibility (same split every time).
- `stratify=y` ensures that the target variable is distributed evenly between the train and test sets (especially important for imbalanced classes).

- ✓ Strategies for Splitting:
 - **Random Split:** Randomly splits the data into training and test sets.
 - **Stratified Split:** Ensures the distribution of the target variable is similar in both training and testing sets. (especially important in imbalanced datasets).
 - **Cross-Validation:** Instead of a single split, cross-validation divides the data into multiple subsets, training and testing the model multiple times on different data, providing a more robust evaluation.

III. Visualizing Data

Data is visualized on Google Colab

IV. Choosing a Model

Random Forest Classifier is used: it is an ensemble learning method that combines multiple decision trees to make better predictions. Here's how it works:

1. Training Multiple Decision Trees:

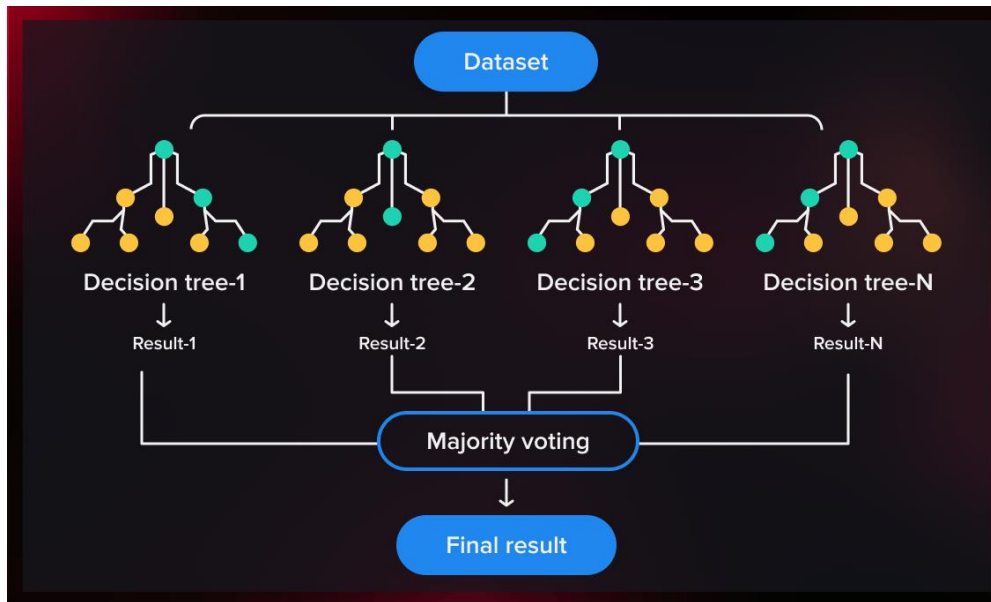
- The model builds several decision trees using random samples of the training data. Each tree is trained on a **different subset** of the data, drawn with **replacement**. This process is called **Bagging**.
- By using different data samples, each tree learns different patterns, reducing the bias of the model.

2. Random Feature Selection:

- When building a tree, **Random Forest** doesn't use all features to split each node. Instead, it randomly selects a subset of features at each split.
- This introduces diversity among the trees and ensures that no single feature dominates the decision-making process, making the model more robust.

3. Prediction (Voting):

- Once all the trees are trained, **Random Forest** makes predictions by letting each tree "vote" on the outcome.
- For **classification** tasks, the class with the most votes is the final prediction (majority voting).



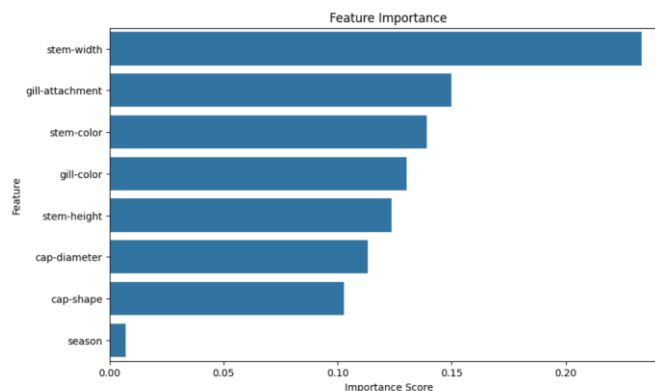
Why Choosing Random Forest?

1. Handling Complex Data Relationships:

- Random Forest can handle complex, non-linear relationships in the data. Our dataset contains various features such as cap shape, cap diameter and gill color, which may interact in complicated ways that linear models might struggle to capture.

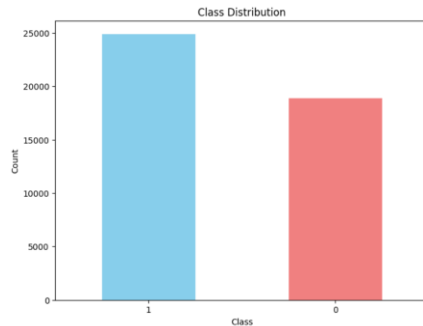
2. Feature Importance:

- One of the key advantages of Random Forest is its ability to compute the importance of each feature. This is useful for understanding which features (such as stem width) is the most influential in determining whether a mushroom is edible or poisonous.



3. Handling Imbalanced Data:

- In our case, the data is slightly imbalanced, and Random Forest can handle imbalanced datasets well. By setting **class_weight='balanced'**, the algorithm adjusts the weights of classes to ensure it doesn't ignore the minority class.
- This prevents overfitting on the majority class and ensures better generalization.



4. Robustness and Overfitting:

- Random Forest is robust against overfitting, especially when the number of trees is large. This is because multiple trees are trained on different parts of the data (subsampling with replacement), and their predictions are averaged to get the final classification.

In our case, the training accuracy and testing accuracy are both high and close to each other, then overfitting is not a problem.

We also tried to use other models such as *Logistic Regression* and *Naive Bayes*

Logistic Regression

Logistic Regression is used to predict the probability of a binary outcome (like "1" or "0"). It works by modeling the relationship between the input features (variables) and the probability of the target class. Logistic regression assumes a linear relationship between the features and the log-odds (the logarithm of the odds) of the target class.

$$\text{logit}(p) = \ln(\text{odds}) = \ln(p/(1-p))$$

$$\text{logit}(p) = \beta_0 + \beta_1 X$$

where p is the probability of the target class being "1", and b_0 , b_1 are the model coefficients (weights) that the model learns.

Since the log-odds are unbounded, the model uses a **sigmoid function** to map the result to a probability between 0 and 1:

$$\frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

The model calculates the probability for each instance. If the probability is greater than a set threshold (0.5), it predicts the positive class 1. If it is less than 0.5, it predicts the negative class 0.

In our case to predict if a mushroom is **edible** (1) or **poisonous** (0), logistic regression will:

- Use the features (like cap diameter, stem color...) to calculate a score (log-odds).
- Apply the sigmoid function to convert that score into a probability.
- If the probability is greater than 0.5, it predicts "edible" (1), otherwise, it predicts "poisonous" (0).

Naïve Bayes

The Naive Bayes classifier is a simple machine learning algorithm used for classification tasks. It is based on Bayes' Theorem, which helps us determine the probability of a class based on certain evidence (features). Bayes' Theorem calculates the probability of a class C given some observed features X.

Formula:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

- $P(C|X)$: This is the posterior probability. It's the probability that a class C is true given the features X we observe.

- $P(X|C)$: This is the likelihood. It's the probability of observing the features X given that we know the class is C .
 - $P(C)$: This is the prior probability. It's the initial probability that class C is true before we know anything about the features X .
 - $P(X)$: This is the evidence. It's the overall probability of observing the features X .
- **Independence Assumption:** The "naive" part of Naive Bayes comes from the assumption that the features are independent of each other. This means that Naive Bayes assumes that knowing the value of one feature doesn't change the probability of another feature.
 - **For each class:** We calculate the probability of each class C based on the features $X_1, X_2, \dots, X_n$ we have. But, because of the independence assumption, instead of calculating for all features together, we simply multiply the probabilities of each feature given the class.

In our case, Naive Bayes is used to classify mushrooms as **edible** or **poisonous**. The dataset would include features like color, cap shape, with the target variable being **class** (edible or poisonous).

- For each feature, the algorithm will calculate the probability of each feature occurring for both the edible and poisonous classes.
- Using Bayes' Theorem, it will calculate the probability that a mushroom is edible or poisonous based on its features.
- The class with the highest probability is chosen as the predicted class

V. Training and Testing

Random Forest code

```
from sklearn.model_selection import train_test_split

# Splitting the data into features (X) and target (y)
X = df.drop(columns=['class']) # Features
y = df['class'] # Target variable

# Splitting the data into a training set (80%) and a testing set (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

rf_model = RandomForestClassifier(random_state=42, class_weight='balanced', max_depth=10)
rf_model.fit(X_train, y_train)

# Predict on the test set
y_pred_rf_model = rf_model.predict(X_test)

# Evaluate the model
classification_rep_rf_model = classification_report(y_test, y_pred_rf_model)
confusion_mat_rf_model = confusion_matrix(y_test, y_pred_rf_model)

# Print the classification report
print("Classification Report:")
print(classification_rep_rf_model)
```

VI. Evaluation

We have measured average accuracy obtained by all three models.
In Logistic regression model we used z-score to normalize data ranges.

1. For Random Forest:

```
Classification Report:
              precision    recall  f1-score   support

     0       0.93      0.98      0.95      3783
     1       0.98      0.94      0.96      4980

 accuracy      0.96      0.96      0.96      8763
 macro avg      0.96      0.96      0.96      8763
 weighted avg      0.96      0.96      0.96      8763

ROC AUC Score: 0.9947545402333627
```

Accuracy of Random Forest :0.96

Random Forest does not require normalization because it is based on decision trees, which are not sensitive to the scale of the features. Since Random Forest

combines multiple decision trees, the scaling of features does not affect the model's performance. Therefore, normalization of features is not necessary when using Random Forest.

2. Logistic regression:

Logistic Regression Classification Report (Standardized):					
	precision	recall	f1-score	support	
0	0.57	0.67	0.62	3836	
1	0.71	0.62	0.66	5093	
accuracy			0.64	8929	
macro avg	0.64	0.65	0.64	8929	
weighted avg	0.65	0.64	0.64	8929	

Logistic Regression ROC AUC Score (Standardized): 0.6883551960643604

Logistic Regression Accuracy:0.64

Logistic Regression may perform poorly if features are not scaled so we do normalization to ensure all features contribute equally to the model. Without it, features with larger values may dominate the learning process. Normalization helps the model converge faster and more reliably by putting all features on the same scale.

3. Naive Bayes:

Naive Bayes Classification Report:					
	precision	recall	f1-score	support	
0	0.58	0.59	0.58	3836	
1	0.69	0.68	0.68	5093	
accuracy			0.64	8929	
macro avg	0.63	0.63	0.63	8929	
weighted avg	0.64	0.64	0.64	8929	

Naive Bayes Accuracy:0.64

Random Forest is the best choice for our case due to its high accuracy, robustness, and flexibility in handling complex data. With an accuracy of 0.96, it outperforms other models, such as Logistic Regression and Naive Bayes, which have much lower accuracy scores. Random Forest excels because it is an ensemble method that builds multiple decision trees, each trained on different subsets of the data, allowing it to capture complex patterns and reduce the risk of overfitting. Random Forest can handle both linear and non-linear relationships, making it highly capable of handling diverse datasets. It also does not require feature scaling, unlike models such as Logistic Regression, and is less sensitive to noise and outliers, making it a highly reliable model for classification tasks like the one at hand.

Logistic Regression performs poorly in our case since it relies on linear decision boundaries to classify data where the relationship between the features and the target variable is complex or non-linear. Logistic Regression will struggle to model this effectively, leading to lower accuracy.

Naive Bayes assumes that all features are independent, meaning it treats each feature as if it has no relationship with the others. In real-world data, features are often correlated, and this assumption is rarely true. When features are related, Naive Bayes fails to capture these dependencies, which can lead to poor performance, as seen in our case. Naive Bayes is a simple model that works well with linear relationships, but it struggles with more complex, non-linear interactions between features. It can't model complex patterns or interactions in the data, which limits its ability to perform well on datasets with more complexity. Resulting in having low accuracy

Based on our results we consider Random Forest to be the best fit model in our dataset case.

VII. Cross Validation

Cross-validation is a technique used to assess how well a model generalizes to unseen data by dividing the dataset into multiple parts (folds). The model is trained on some parts and tested on the remaining part. This process is repeated several times to ensure that each fold of the data is used for both training and testing. The most common form is K-fold cross-validation, where the data is split into K subsets or "folds," and the model is trained and evaluated K times, each time using a different fold as the test set.

 Cross-validation accuracy scores for each fold: [0.95946511 0.96002229 0.9568185 0.95959877 0.96322095]
Average cross-validation accuracy: 0.9598251238257959
Standard deviation of cross-validation accuracy: 0.002039174916965911

How It Works:

- **Data Splitting:** The dataset is divided into K equally sized subsets (or folds).
- **Training and Testing:** For each fold, the model is trained using K-1 folds, and the remaining fold is used for testing.
- **Performance Metrics:** The accuracy is calculated for each fold. The model's overall performance is averaged over all the folds to get an estimate of its generalization ability.

The average accuracy of **95.98%** shows that the model is performing well, correctly predicting the target variable in about 96% of cases. This is a strong indication that the model is likely to perform well on new, unseen data.

The standard deviation of the cross-validation accuracies is **0.0020**, which is very low. This indicates that the model's performance is stable and does not fluctuate much across different data splits. A low standard deviation suggests that the model generalizes well and is not overfitting to any particular fold of the data.

VIII. Hyperparameter Tuning

Hyperparameter tuning is the process of optimizing the hyperparameters of a machine learning model to improve its performance.

We have chosen the Random Forest Classifier for our dataset based on the results obtained so far. Next, we will explore how adjusting various model parameters impacts its accuracy. The parameters under consideration for tuning in this model are as follows:

```
param_dist = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [10, 15, 20, None],  
    'min_samples_split': [2, 5, 10],  
    'min_samples_leaf': [1, 2, 4],  
    'max_features': ['sqrt', 'log2'],  
    'class_weight': ['balanced']  
}
```

Parameters:

n_estimators: The number of trees in the forest.

- **Effect:** Increasing the number of trees generally improves the model's accuracy but it also increases computational time. A smaller number of trees may make the model faster but less accurate.

max_depth: The maximum depth of each decision tree.

- Effect: Deeper trees can capture more complexity in the data but may lead to overfitting. Shallower trees may underfit the data, resulting in lower accuracy.

min_samples_split: The minimum number of samples required in a leaf node.

- Effect: Smaller values allow the model to create more leaf nodes, which may overfit the training data. Larger values force the model to create fewer, larger leaves, which can prevent overfitting

max_features: The number of features to consider when looking for the best split.

- Effect: Restricting the number of features (sqrt or log2) can improve model robustness and reduce overfitting. Using more features can increase accuracy for training but may risk overfitting on smaller datasets.

class_weight: Adjusts weights for classes based on their frequencies in the data.

- Effect: Helps the model handle imbalanced datasets by giving more importance to the minority class, ensuring better generalization for underrepresented classes.

We use **Randomized Search** instead of **Grid Search** for hyperparameter tuning, especially with our large datasets, because it is more efficient in exploring the parameter space. Grid search exhaustively tests all possible combinations of hyperparameters, which can be computationally expensive and time-consuming, particularly with a large dataset and many hyperparameters. In contrast, Randomized Search randomly samples from the parameter grid, allowing it to find good hyperparameters faster and with fewer evaluations. This makes Randomized Search a better choice for large datasets, where computational resources and time are limited, while still providing high-quality results.

Result:

```
from sklearn.model_selection import RandomizedSearchCV
import numpy as np

param_dist = {
    'n_estimators': [50, 100, 200],
    'max_depth': [10, 15, 20, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2'],
    'class_weight': ['balanced']
}

# Initialize the Random Forest model
rf_model = RandomForestClassifier(random_state=42)

# Perform Randomized Search with 3-fold cross-validation
random_search = RandomizedSearchCV(estimator=rf_model, param_distributions=param_dist, n_iter=10, cv=3, n_jobs=-1, scoring='accuracy', random_state=42)

# Perform the randomized search
random_search.fit(X_train_scaled, y_train)

# Print the best parameters and best score
print("Best Parameters from Randomized Search:", random_search.best_params_)
print("Best Cross-validation Accuracy from Randomized Search:", random_search.best_score_)

# Get the best accuracy from RandomizedSearchCV
best_accuracy_random = random_search.best_score_
print(f"Best Accuracy from Randomized Search: {best_accuracy_random}")
```

```
Best Parameters from Randomized Search: {'n_estimators': 50, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth': 20, 'class_weight': 'balanced'}
Best Cross-validation Accuracy from Randomized Search: 0.9871283861334664
Best Accuracy from Randomized Search: 0.9871283861334664
```

This suggests that the Random Forest classifier performs optimally with the parameters highlighted in the image above.

IX. Validation Testing

To assess whether the model can accurately predict unseen data, we used the `best_rf_model` with the best parameters identified in the previous section. This model was tested on data it had not been trained on to evaluate its performance and ensure that underfitting is not occurring.

```
import pandas as pd
from sklearn.metrics import classification_report, roc_auc_score

# Step 1: Load the new dataset
df_new = pd.read_csv('Generated_Dataset.csv')
X_new = df_new.drop(columns=['class'])
y_new = df_new['class']

y_pred_best_rf = best_rf_model.predict(X_new)

# Step 5: Generate the classification report
classification_rep_best_rf = classification_report(y_new, y_pred_best_rf)
print("Best Random Forest Classification Report:")
print(classification_rep_best_rf)

# Step 6: Calculate the ROC AUC score for the best model
y_pred_prob_best_rf = best_rf_model.predict_proba(X_new)[:, 1]
roc_auc_best_rf = roc_auc_score(y_new, y_pred_prob_best_rf)
print(f"Best Random Forest ROC AUC Score: {roc_auc_best_rf}")
```

```

➔ Best Random Forest Classification Report:
      precision    recall  f1-score   support

     0       0.98      0.96      0.97        51
     1       0.96      0.98      0.97        49

 accuracy          0.97        100
  macro avg       0.97      0.97      0.97        100
 weighted avg     0.97      0.97      0.97        100

```

Best Random Forest ROC AUC Score: 0.9983993597438975